

DuckDB as backend to build optimization models in JuMP.jl

Abel S. Siqueira, Netherlands eScience Center
Diego A. Tejada Arango, TNO

Use DuckDB to solve sparse indices in JuMP and separate indices creation from model creation.

Motivation

In the development of TulipaEnergyModel.jl, we wanted to avoid some performance pitfalls when some indices in a JuMP model depend on others, e.g.,

```
@variable(model, x[a in A, b in B[a]])
```

In particular, we try to avoid the “sum-if problem” [1] in the constraints and the sparse indices storage. Additionally, we wanted to decrease data movement when transforming input data into model data.

Approach

We use DuckDB to store the variables and constraints indices of our model as tables. This allows for an efficient local data storage that scales well and that fits nicely in a larger data pipeline. Although our implementation is tightly coupled to our application, we try to present it here in a generic way.

The steps below could be done with a single DuckDB memory-only connection from beginning to end, or in multiple sessions storing the DuckDB DB locally. We show a general visualisation in Figure 1.

1. **Load input tables.** Create a DuckDB connection and load your data into it.
2. **Create indices table.** Using your favorite DuckDB client, create the indices of variables, constraints, expressions, and other helper tables.
3. **Create model.** Create variables and constraints looping over the rows of their corresponding indices.
4. **Solve model.** Solve the model, get the solution, and save it in DuckDB.
5. **Export.** Export to files, create figures, connect to other DBs, to dashboards, etc.

References

[1] JuMP docs for the sum-if problem



[2] Discourse post about using DataFrames



[3] “JuMP, GAMS, and the IJLKM model” blogpost



[4] This poster and its code



Indices tables

The indices tables are the main feature of the approach. Instead of storing dynamic indices using dictionaries or other sparse structures, we store them in a linearised way in tabular format.

The idea came from a suggestion to use DataFrames to compute incoming flow in a node over different time blocks [2] in order to avoid the “sum-if problem” [1].

Soon after, we decided to use DuckDB as input and realized we were converting tabular data into Julia structures and back into tabular data to create the JuMP models. We decide to try to use DuckDB from beginning to end.

Figure 2 shows our storage approach. Each DuckDB table has 3 parts. The indices form a primary key to the table, but we also attach a numeric id for reference. Finally, we can attach relevant data, such as lower bounds for variables and right-hand-side values for constraints. This extra data could be stored in the table itself or joined with the data only during model creation.

The same storage model is used for variables, expressions and constraints. The ids of variables, expressions, and possibly their coefficients, can be aggregated to use during model creation. An example is shown in Code 1, where we compute the DuckDB table for the following constraint:

$$\sum_{a \in A: b \in B_a} x_{a,b} \leq p_b \quad \forall b \in B = \bigcup_{a \in A} B_a$$

In the Code, `var_x` is a table containing columns `id`, `a`, and `b` equivalent to the variable `x` from the Motivation. We compute the id of the cons, and use `b` is the index of this constraint. We also store the aggregate `x` indices per `b`, and a value `p`.

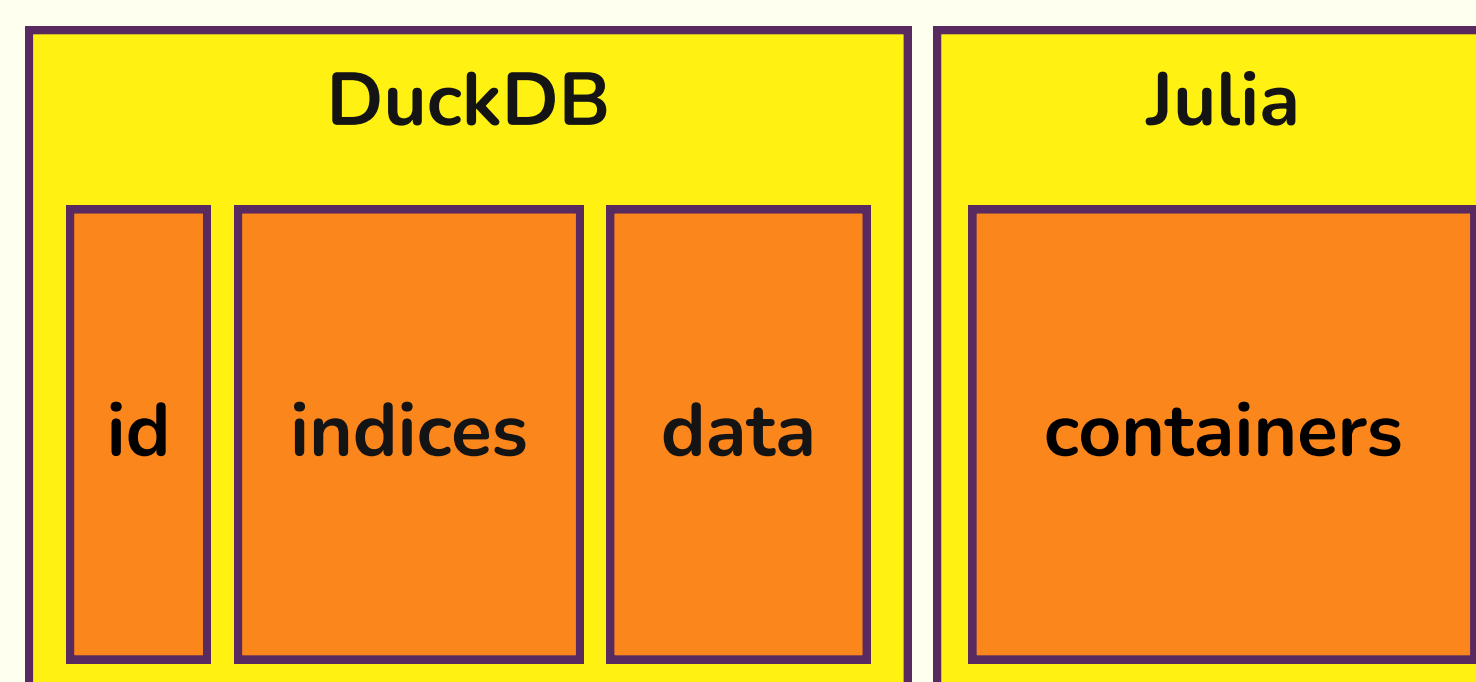


Fig. 2: Indices and containers

Create model

Once the DuckDB tables are created, we must create the Julia containers.

Each container is a Vector of JuMP variables, constraints or expressions. This containers must be tied to a table such that the row id corresponds to the container index.

In Code 2, we create the JuMP variables according to the same `var_x` table as Code 1. The `base_name` is used to give it a nice display name.

In Code 3, we create the JuMP constraints corresponding to the table `cons` created in Code 1. We aggregated the indices of `x`, and now we sum over them. The `p` parameter stored in Code 1 is now used in the right-hand-side.

Comparison

We use the IJLKM problem [3] to compare DataFrames- and DuckDB-based solutions. The code is available in [4]. Figure 3 shows the comparison. This problem is mostly about avoiding the sum-if problem with a double inner join, which makes the strategies comparable in time. On the other hand, DuckDB uses less memory, while allowing more flexibility in data management and a better separation of concerns.

```
CREATE SEQUENCE cons_id START 1;
CREATE TABLE cons AS
WITH cte_x AS (SELECT var_x.b,
    ARRAY_AGG(var_x.id) AS x_ids,
    FROM var_x GROUP BY var_x.b
) SELECT nextval('cons_id') AS id,
    data_per_b.b, cte_x.x_ids, data_per_b.p
FROM data_per_b
LEFT JOIN data ON cte_x.b = data_per_b.b;
```

Code 1: Constraints indices with variable aggregation. Here,

```
sql(s) = DuckDB.query(connection, s)
var_name(row) = "x[a=${row.a},b=${row.b}]"
x = model[:x] = [@variable(model,
    base_name = var_name(row)
) for row in sql("FROM var_x")]
```

Code 2: Variable creation with DuckDB

```
model[:cons] = [@constraint(model,
    sum(x[id] for id in row.x_ids) <= row.p
) for row in sql("FROM cons")]
```

Code 3: Constraints creation with DuckDB

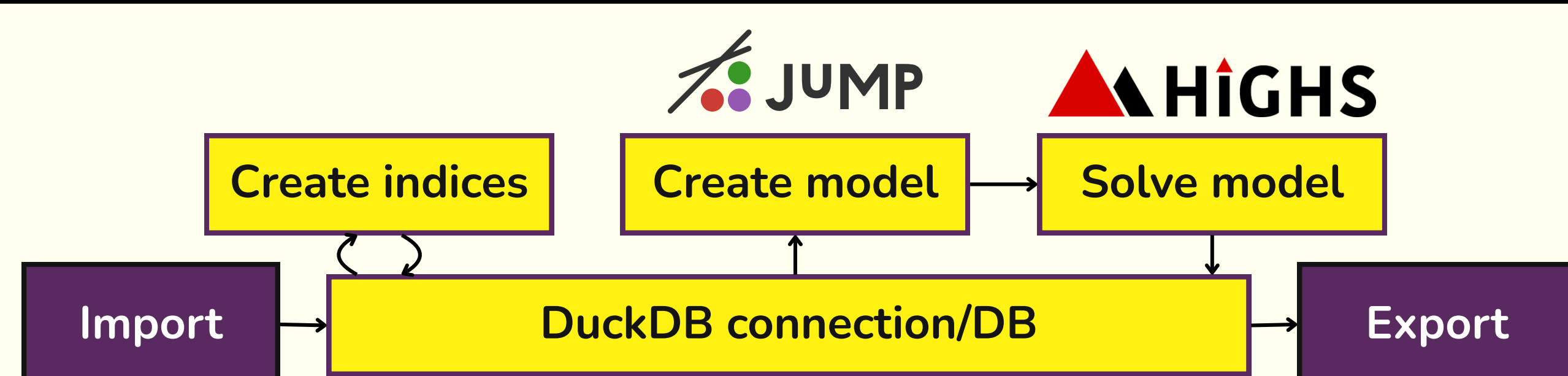


Fig. 1: DuckDB-based pipeline

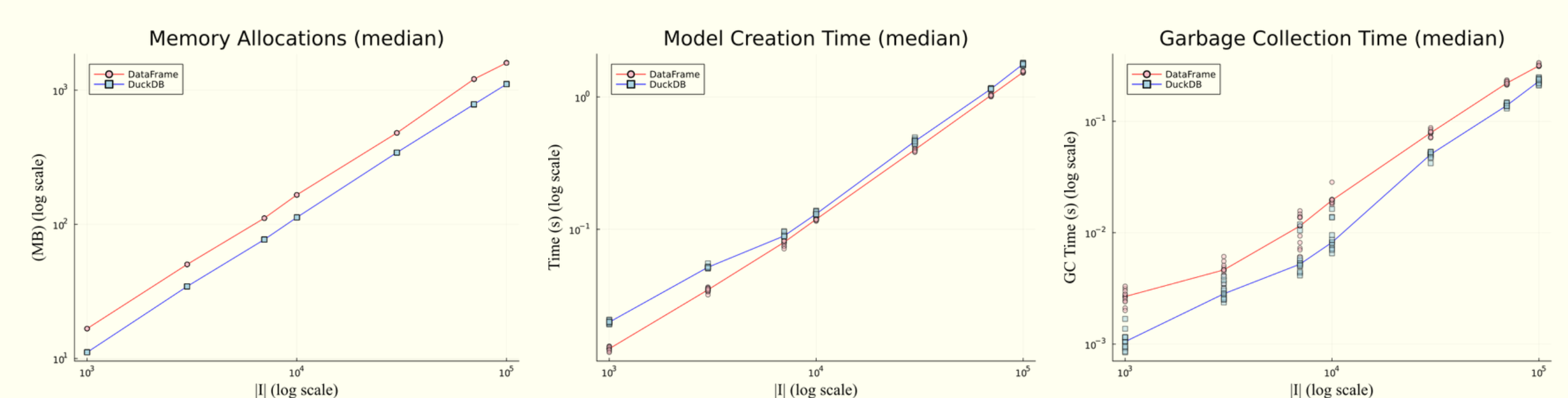


Fig. 3: Time, GC time, and memory comparison on the IJLKM problem [3]

Acknowledgements

We thanks Oscar Dowson the suggestion to use DataFrames [2] that motivated this DuckDB backend. This publication is part of the project NextGenOpt with project number ESI.2019.008, which is (partly) financed by the Dutch Research Council (NWO) and supported by eScienceCenter under project number NLeSC C 21.0226.

netherlands

eScience center

TNO innovation for life

